

PROJECT NOTES — PART 2

SkyGuard AI

Intelligent Real-Time Anomaly Detection for Automatic Weather Stations
— AI/ML Development Log —

Smart India Hackathon • AI/ML Core Track

Sanchal Kumar • Sagar Institute of Research & Technology, Bhopal

Continues from Part 1 (Physics Rule Engine) → LSTM Autoencoder → Dataset Refinement → Spatial Engine Groundwork

7. Building the LSTM Autoencoder

With the Physics Rule Engine in place, the next milestone was the LSTM Autoencoder — the layer meant to catch anomalies that only reveal themselves across several days, not in a single reading. The idea: train a model to reconstruct a 7-day sequence of `(avg_temp, air_pressure, relative_humidity)` for one station, showing it only NORMAL sequences during training. It never learns what a fault looks like — it only learns what normal looks like. At test time, I feed it every kind of sequence and measure the reconstruction error. On normal data the error stays low; on an anomalous sequence — a spike, a stuck sensor, a drift — the pattern doesn't match anything it learned, so the error spikes. That error becomes the anomaly score.

Getting the Timeline Ready First

Before I could build 7-day sequences, I had to deal with two realities of the data: short gaps in `air_pressure` (a handful of missing days here and there), and occasional multi-day network-wide outages I found when I checked — three separate windows where nearly all 14 stations went silent at once, most likely a shared data-collection failure rather than independent station problems.

I built `sequence_prep.py` to handle this in three steps: interpolate only the short gaps (3 days or fewer — a linear fill is trustworthy at that length, but I left longer gaps as missing rather than fabricate a week of fake data); split each station's timeline into continuous "chunks," breaking wherever there's a real calendar gap or a gap too long to interpolate; and only ever slide a 7-day window within a single chunk, so a sequence can never silently span a real discontinuity.

A bug I found while testing this: when a row with missing data started its own isolated chunk, the very next valid row was silently continuing into that same chunk instead of starting fresh — leaving a few NaN values inside otherwise-valid sequences.

How I fixed it: rows with missing data are now excluded from the eligible timeline entirely, rather than merely flagged — so both the row before and the row after a gap correctly see a break. I re-verified afterward: zero NaNs remained in any sequence.

Splitting by Date, Not Randomly

I split sequences chronologically rather than shuffling randomly — training on data before 2023-11-17, validating on 2023-11-17 to 2024-06-29, and reserving everything after 2024-06-29 as a genuinely unseen test set. A random split would put near-identical neighbouring days on both sides of the split, letting the model effectively "see" data it's supposed to be tested on. Only training and validation are restricted to NORMAL sequences — the test set includes every anomaly type, since that's the only honest place to report detection performance.

Results

Trained for 350 epochs — the loss was still improving at the final epoch, so this run hadn't fully plateaued either, but the gains per epoch were small by that point. On the held-out test set:

Anomaly Type	Caught by LSTM	Notes
SUDDEN_SPIKE	77.9%	Strong — a clear one-day deviation from the learned pattern
GRADUAL_DRIFT	64.7%	Good — this is exactly the kind of pattern an LSTM should catch
REGIONAL_HEAT_EVENT	23.3%	Expected — this model has no spatial context, so it partially confuses a real regional event for a deviation
REGIONAL_PRESSURE_DROP	21.6%	Same reason as above
STUCK_AT	3.0%	Unexpectedly weak — see below

False positive rate on NORMAL test rows: 0.1%.

The STUCK_AT surprise: I expected the LSTM to be particularly good at catching a frozen sensor, since that's a multi-day pattern. Instead it was the weakest result, even after full training. The reason makes sense once you think about it: a flat, unchanging sequence is actually easy for an autoencoder to reconstruct, because the model has already learned "tomorrow looks like today" as a general rule from normal data. A stuck sensor just follows that rule more perfectly than usual — so the model doesn't see it as unusual at all.

8. Fixing the STUCK_AT Blind Spot

Since neither the Physics Engine (3.0%) nor the LSTM (3.0%) could catch a frozen sensor reliably, I built a dedicated, deterministic check for it rather than hoping a later layer would somehow pick up the slack.

Why a 7-Day Window Wasn't the Right Tool

My first idea was to check the 7-day rolling standard deviation — near-zero std should mean "nothing is changing." But when I checked the actual numbers, this barely separated stuck sensors from normal ones. The reason: my injected stuck-runs last 4–8 days, so a 7-day window usually only *partially* overlaps a real stuck period, mixing real values with the frozen value and never looking "flat enough" to stand out.

Switching to a much shorter window — 3 days — fixed this. A short window is far more likely to land entirely inside even a short stuck-run. I checked the actual std distributions before picking a threshold rather than guessing one: normal readings sit comfortably above 0.1–0.3 in 3-day rolling std, while stuck sensors cluster right at 0.000. I set the threshold at 0.05.

Result

Detector	STUCK_AT Recall
Physics Rule Engine	3.0%
LSTM Autoencoder	3.0%
New 3-day rolling-std rule	67.1%

False positive rate on NORMAL rows: 0.59%. I built this as its own standalone module (`stuck_at_check.py`) rather than folding it into an existing file, since it's meant to be consumed as one more independent signal by the Isolation Forest and Fusion layers later — not a final verdict by itself.

9. A Geography Problem in My Own Dataset

While starting to design the Spatial Correlation Engine — the layer that checks whether nearby stations agree, to tell a real regional event apart from an isolated sensor fault — I needed to define "nearby" properly for the first time. That's when I found a real problem in how I'd generated the dataset months earlier.

What I Found

My regional-event injector had been grouping stations by their `cluster_label`, not by actual distance. One of those labels — "Isolated Edge-Case" — contained both Bikaner and Car Nicobar, which are **1,740 km apart** (mainland Rajasthan to an island territory). My injector had been treating them as one weather region and applying synchronized "regional" heatwaves and pressure drops to both at once — something no real weather system could ever do.

How I Fixed It

I rewrote the grouping logic to use actual haversine distance between station coordinates (150 km cutoff) instead of the label. That fixed the Bikaner/Car Nicobar problem, but testing it immediately surfaced a second, subtler issue: a distance-only rule still grouped Shimla (a Himalayan hill station at 1,546 m elevation) with Punjab plains stations 83 km away at around 270–350 m. Horizontally close, but climatically very different — hill stations and plains don't move together during a regional weather event the way two plains stations would.

The final rule: two stations only count as one "region" if they're within 150 km *and* within 400 m of elevation of each other. Both conditions matter — distance alone isn't enough. Under this rule, Bikaner, Car Nicobar, and Shimla are all correctly excluded from ever receiving a genuine regional event now — they simply have no real geographic neighbor in this 14-station network, which is the physically honest outcome.

Regenerating and Re-Verifying Everything

Fixing this meant regenerating the dataset, which meant every downstream number I'd already reported — Physics Engine, LSTM, STUCK_AT check — needed to be re-run and re-verified against the new data, not just assumed to still hold. I did that, on my own machine, and confirmed every number reproduced correctly:

Check	Result on the corrected dataset
Dataset shape / category counts	NORMAL 19,539 · SENSOR_FAULT 816 · GENUINE_EVENT 477
Physics Engine — false positive rate	0.48%
Physics Engine — genuine events wrongly flagged	0.0–0.4% (correctly low)
STUCK_AT check	67.1% recall, 0.59% false positive — unchanged, as expected
LSTM Autoencoder (retrained)	Consistent with the numbers in Section 7 (77.9% / 64.7% / 3.0%)

Nothing broke, and nothing needed to be re-designed — only re-run against corrected data. That in itself told me the layers really are independent of each other, the way they were meant to be.

10. Where I Am Right Now

Phase	Status
Problem definition & scope	DONE
Dataset design (10 features, 3 model inputs)	DONE
Synthetic anomaly injection pipeline	DONE
Dataset validation & geography fix (5 rounds total)	DONE
Physics Rule Engine	DONE
LSTM Autoencoder (temporal patterns)	DONE
STUCK_AT dedicated fix	DONE
Spatial Correlation Engine	STARTING NEXT
Isolation Forest (multivariate outliers)	NEXT
Fusion Layer + Confidence Scoring	NEXT
Explainability (SHAP)	NEXT

What Comes Next

The Spatial Correlation Engine is up next, and it will directly reuse the distance-and- elevation neighbor logic I just built while fixing the dataset — just applied in the opposite direction. Instead of asking "which stations should move together when I inject an event," it will ask, for a real incoming reading, "did this station's neighbors show the same pattern, or is this one isolated?" That's the piece that will finally let the system tell a sensor fault apart from a genuine regional event with real evidence, instead of the LSTM's partial, context-free guess.